



World Choice Perfumes

Comprehensive System Specification

A Production-Grade Multi-Branch Perfume Business Management
System

Built with Laravel, Supabase & Cloudinary

Version: 1.0

Date: August 21, 2026

Status: System Specification (Pre-Development)

Framework: Laravel (PHP)

Database: Supabase (PostgreSQL)

Storage: Cloudinary

Business Contact Information

Platform	Handle / Number
Instagram	@world_choice_parfum
Phone / WhatsApp	+255 710 603 637
TikTok	Kessy Mohammed02

Note: These details will appear in the website's *Contact Us*, *Footer*, *About*, and *Social Media* sections of the World Choice Perfumes public-facing website.

Table of Contents

1. Project Overview

1.1 Business Description

1.2 System Purpose

1.3 Core Design Principles

2. Actors & Roles

2.1 Super Admin

2.2 Branch Admin

2.3 Cashier

2.4 Customer

3. Authentication & Registration

3.1 Super Admin Registration

3.2 Branch Admin Registration

3.3 Cashier Registration

3.4 Customer — No Registration

3.5 OTP Verification

3.6 Account States

4. Branch Management

4.1 Branch Creation

4.2 Branch Geolocation

4.3 Branch Isolation

5. Product & Stock Management

5.1 Products

5.2 Stock Entry

5.3 Stock Movements

5.4 Branch-Specific Stock

6. Sales & Transactions

6.1 Cashier Sales Process

6.2 Customer Information Capture

6.3 Transaction Audit Trail

6.4 Automatic WhatsApp Receipt

7. Customer Ordering System

7.1 Public Browsing

7.2 Branch-Based Ordering

7.3 Order Workflow & Locking

7.4 Order States

7.5 Future Payment Integration

8. Cashier Accountability

8.1 Accountability Report

8.2 Discrepancy Reasons

8.3 Cashier Expenses

9. Financial Management

9.1 Profit Calculation

9.2 Expense Management

9.3 Financial History

10. Reporting

10.1 Report Types

10.2 Reporting Scope

11. User Profile Management

11.1 Super Admin Profile

11.2 Branch Admin Profile

11.3 Cashier Profile

12. Technology Stack

12.1 Core Technologies

12.2 External Services

12.3 Architectural Principles

13. System Architecture

13.1 High-Level Architecture

13.2 Organizational Hierarchy

13.3 Data Isolation Model

14. Database Design Requirements

14.1 Core Entities

14.2 Key Relationships

15. Security Requirements

16. Business Rules Summary

17. Future Enhancements

18. Appendices

A. Registration Flow Diagrams

B. Order Workflow Diagram

C. Permissions Matrix

1. Project Overview

1.1 Business Description

World Choice Perfumes is a multi-branch perfume retail business operating in Tanzania. The company sells perfumes through physical retail branches across multiple cities, with plans for online ordering and delivery. The business manages inventory, sales, staff, and finances across all branches from a centralized digital system.

1.2 System Purpose

This system specification defines a **production-grade web application** that will:

- Manage all branches of World Choice Perfumes from a single platform
- Provide role-based access for Super Admins, Branch Admins, Cashiers, and Customers
- Track inventory, sales, expenses, and profit across all branches
- Enable public-facing product browsing and order placement
- Generate automated WhatsApp receipts after sales
- Provide real-time cashier accountability and loss tracking
- Generate comprehensive business reports
- Support future payment API integration

1.3 Core Design Principles

Principle 1 — Business Rules First: Every architectural decision must serve a real business rule. No feature exists merely to appear sophisticated.

Principle 2 — Data Integrity Above All: Database-level enforcement of business rules. Concurrency protection, atomic operations, and transactional safety are mandatory.

Principle 3 — Branch Isolation by Default: Each role sees only the data it is authorized to access. This is enforced at the backend/database level, not

merely by hiding UI elements.

Principle 4 — Loose Coupling with External Services: External integrations (WhatsApp, payments, maps) are designed as swappable modules so providers can change without rewriting the core system.

Principle 5 — Maintainable, Professional Code: Clean Laravel architecture, proper conventions, tested code, and documented decisions.

2. Actors & Roles

The system has **four distinct actors**, each with a clearly defined scope of access and responsibilities.

Actor	Scope	Authentication	Profile Picture	GPS Location
Super Admin	Entire company, all branches	Email OTP 	Not required 	Not applicable 
Branch Admin	One assigned branch only	Email OTP 	Business/branch photo 	Branch GPS coordinates 
Cashier	Operational records within one branch	Email OTP 	Personal photo 	Uses branch location 
Customer	Public browsing & ordering	No login required 	N/A	Phone GPS (for directions) — not stored

2.1 Super Admin

The Super Admin is the **highest-level administrator** with company-wide control.

Responsibilities

- Manage and monitor **all branches**
- Manage and monitor **all Branch Admins**
- View **company-wide** sales, stock, expenses, profits, and reports
- View **cross-branch** activity and performance
- Create, edit, and deactivate branches
- Assign Admins to branches
- Access company-wide audit logs
- Manage system-wide settings

Restrictions

- Does **not** register a physical location
- Does **not** need a profile picture per current design
- Must verify email using OTP during registration

2.2 Branch Admin

The Branch Admin manages **one specific branch** and sees only that branch's data.

Example: Admin John → Kariakoo Branch

John can see Kariakoo sales, stock, cashiers, expenses, and reports.

John **cannot** see Mbezi sales, Arusha expenses, or Mwanza stock — unless the Super Admin grants broader permission.

Responsibilities



Profile Management

- View own profile
- Edit name, phone number, email
- Change profile picture
- Update other allowed personal information
- Change own password
- **Cannot** see cashier passwords



Cashier Management & Monitoring

- View cashiers belonging to own branch
- Approve or reject pending cashier registrations
- Monitor who sold what
- View sales by each cashier
- View sales for today and sales history
- View revenue generated by each cashier
- View profit attributable to transactions/cashiers
- Monitor refunds, voids, and cancellations
- Track cashier activity



Stock Management

- Enter new stock into the branch

- Define product details (name, buying cost, selling price)
- Monitor stock levels
- Track stock movements
- Record supplier/purchase information
- Track date received and person who entered stock

Financial Management

- View profit and expense history for own branch
- Select date ranges (daily → weekly → monthly → yearly)
- View total sales, cost of goods sold, gross profit, expenses, net profit
- View number of transactions and products sold
- Monitor cashier performance
- Track stock received and stock remaining

Reports

- Daily, weekly, monthly, yearly reports
- Sales, profit, expense, stock reports
- Cashier performance reports
- Product performance reports
- **All reports are restricted to own branch only**

Branch Settings

- Upload business/branch profile picture
- Register the branch's geographical coordinates (latitude/longitude)
- Edit business profile information

2.3 Cashier

The Cashier operates at the **branch level** under the authority of the Branch Admin.

Registration & Approval

Cashier Registers → Selects Branch → Email OTP Verification → Account
Status: PENDING
→ Branch Admin Reviews → Approve or Reject

PENDING → Admin Approves → APPROVED → Can Login

PENDING → Admin Rejects → Removed/Rejected

PENDING → Admin Does Nothing → Login Remains Prohibited

Responsibilities

Sales

- Make sales using system-determined prices (cannot change selling price)
- Enter customer information (name, WhatsApp/phone, optional email)
- Select products and quantities
- Commit sales — triggering automatic stock reduction and receipt generation

Expenses

- Create expenses with detailed descriptions (electricity, water, transport, cleaning, packaging, etc.)
- **Must** provide a detailed explanation for every expense
- Expenses are tracked and visible to the Admin with the cashier's attribution

Profile

- View and edit permitted personal information (name, phone, profile picture, email)
- **Cannot** change branch assignment, approval status, or role

2.4 Customer

The Customer is a **public user** who does not need an account to browse the website.

Capabilities

- Browse products by selecting a specific branch
- See products available at each branch with prices
- Get navigation directions to a branch using Google Maps
- Place orders (without payment in Phase 1)
- Provide contact information for order fulfillment

Two Types of Location

Branch Location (Stored): Latitude and longitude of the physical branch, set by the Admin. Persistent in the database.

Customer's Current Location (Not Stored): The customer's phone GPS provides their current position to Google Maps for navigation. This is **not** stored in the system database.

3. Authentication & Registration

3.1 Super Admin Registration

The Super Admin registers at the top level and must verify their email.

1. Enter name, email, phone, and company secret code
2. System sends OTP to the email address
3. Admin enters OTP
4. If `Entered OTP == Generated OTP` → Account activated
5. Otherwise →  Invalid OTP

3.2 Branch Admin Registration

Admin registration follows a **3–4 step workflow**:

Step 1 — Admin + Branch Information

- Admin name
- Phone number
- Email address
- Branch name
- Company secret code
- Other required information

→ Click **NEXT**

Step 2 — Email OTP Verification

- System sends OTP to the entered email
- Admin enters OTP
- If valid → continue; otherwise → error message

Step 3 — Business Profile (Optional)

- Upload business/branch profile picture — **or skip**

Step 4 — Track Business Location (Optional)

- When clicked: *"Are you sure you are physically at your assigned branch?"*
- If **Yes**: Obtain latitude/longitude and store against the branch
- If **Skip**: Allowed. Admin can add/change location later through profile/branch settings

Design Note: GPS coordinates should not be permanently dependent on the registration process. The Admin can update them at any time.

3.3 Cashier Registration

1. Cashier enters personal information
2. Selects a branch to join
3. Verifies email via OTP
4. Uploads a profile picture (optional)
5. Account status becomes **PENDING**
6. Branch Admin reviews and approves or rejects

Important: A cashier in PENDING state **cannot log in** even though their credentials exist in the database. Login is blocked until approval.

3.4 Customer — No Registration

Customers are **public users**. They do not create accounts, do not log in, and do not have passwords. They browse products, place orders, and provide contact information during the ordering process.

3.5 OTP Verification

All internal accounts (Super Admin, Branch Admin, Cashier) require **email OTP verification** during registration. The OTP system is company-wide and consistent across all three roles.

3.6 Account States

State	Description	Login Allowed
PENDING	Registered but awaiting Admin approval (Cashier only)	 No
APPROVED	Active and authorized	 Yes
REJECTED	Removed or denied by Admin (Cashier only)	 No
ACTIVE	Normal active state (Super Admin, Branch Admin)	 Yes

4. Branch Management

4.1 Branch Creation

Branches are created by the **Super Admin**. Each branch has:

- Branch name
- Physical address
- Assigned Branch Admin
- Geographic coordinates (latitude/longitude)
- Business profile picture
- Status (active/inactive)

4.2 Branch Geolocation

Each branch stores its **latitude and longitude**. This is used for:

- Customer navigation — "Get Directions" opens Google Maps
- Branch identification on maps
- Potential future proximity features

Rule: Branch coordinates are stored once by the Admin and reused by every customer. The customer's phone provides their current location to Google Maps — the system does **not** need to permanently track customer locations.

4.3 Branch Isolation

This is the **most fundamental security rule** in the system:

-  **Super Admin** = sees entire company (all branches)
-  **Branch Admin** = sees only assigned branch
-  **Cashier** = sees only own operational records within assigned branch
-  **Customer** = sees public product info for selected branch

This isolation is enforced at the **backend/database level**, not merely by hiding menu items in the frontend.

5. Product & Stock Management

5.1 Products

Products are defined at the company level but stocked and priced at the branch level.

Product Information

- Product name (e.g., "Dior Sauvage")
- Description
- Brand
- Category
- Product images (stored on Cloudinary)
- Status (active/inactive)

5.2 Stock Entry

The Branch Admin enters new stock for their branch. Each stock entry includes:

Field	Description	Example
Product	The perfume being stocked	Dior Sauvage
Quantity	Number of units received	20
Buying Cost	Cost per unit from supplier	TZS 150,000/unit
Selling Price	Expected selling price per unit	TZS 220,000/unit
Supplier	Supplier/purchase information	Supplier name/details
Date Received	When stock was received	August 20, 2026
Entered By	Person who entered the stock	Admin John

5.3 Stock Movements

The system must distinguish between and track:

- **Stock entry** — new stock arriving
- **Stock sale** — stock reduced by a sale
- **Stock return** — customer return restoring stock
- **Stock adjustment** — manual correction
- **Damaged/missing stock** — potential cashier-attributed loss

5.4 Branch-Specific Stock

The public product system understands:

Product + Branch + Stock + Price + Availability

Each branch maintains its own stock levels and can have different selling prices for the same product. A customer browsing the Kariakoo branch sees Kariakoo's stock and prices; browsing Mbezi shows Mbezi's stock and prices.

6. Sales & Transactions

6.1 Cashier Sales Process

Once approved, a cashier can make sales. **The cashier cannot change the selling price.**

```
Cashier selects customer → Adds products to cart → System calculates  
total  
→ Cashier commits sale → System validates → Records transaction  
→ Reduces stock → Generates receipt → Sends WhatsApp receipt to  
customer
```

Price Integrity

Critical Rule: The selling price is set by the Branch Admin. The cashier sees the system-determined price and cannot modify it.

Example: Dior Sauvage → TZS 200,000. If customer wants 2 units:

$$2 \times 200,000 = 400,000$$

The cashier cannot change it to $2 \times 50,000$ unless a future business rule specifically allows authorized discounts.

6.2 Customer Information Capture

During each sale, the cashier enters:

- Customer name
- WhatsApp/phone number (for receipt delivery)
- Email — optional
- Products and quantities

This information is stored for future use, building a customer purchase history.

6.3 Transaction Audit Trail

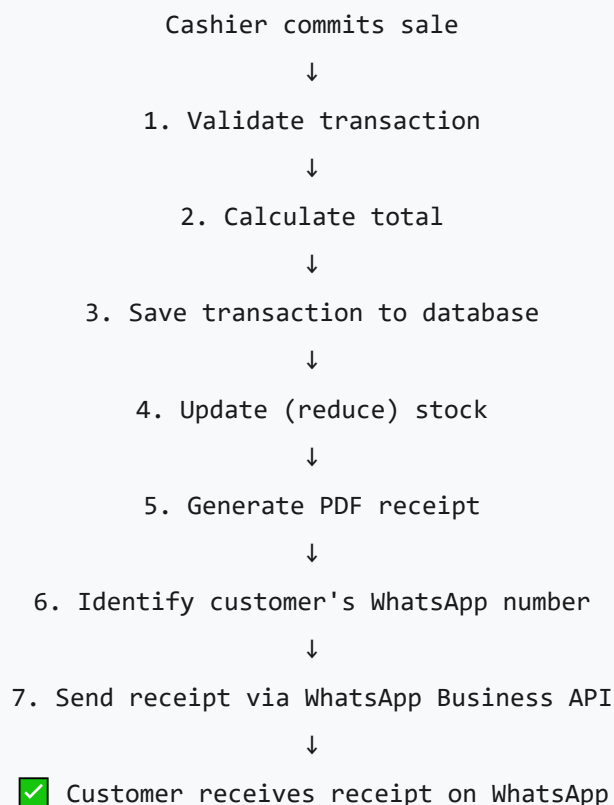
Every sale must have a complete audit trail:

Who sold → What was sold → How much → At what price → When → Which branch → Which customer

Once committed, the sale is recorded with all relevant data: cashier, branch, products, quantities, prices, total, timestamp, and customer information.

6.4 Automatic WhatsApp Receipt

Immediately after a cashier commits a sale, the system performs the following sequence:



Implementation Note: This requires a **WhatsApp Business API integration**. An ordinary personal WhatsApp account cannot automatically send arbitrary PDFs. The specific provider/API will be decided during implementation.

7. Customer Ordering System

7.1 Public Browsing

Customers browse products without logging in. They can:

- Select a branch to view its products
- See product details, images, and prices
- Check product availability
- Switch between branches to compare offerings

7.2 Branch-Based Ordering

Orders are **branch-specific**. When a customer places an order, it is associated with a particular branch.

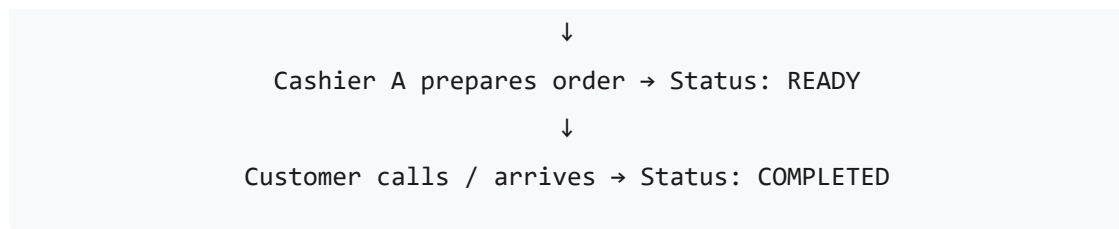
Customer Order Information

- Selected branch
- Products and quantities
- Customer name
- WhatsApp/phone number
- Delivery notes (if applicable)

7.3 Order Workflow & Locking

All approved cashiers belonging to the branch can see pending orders. **Order locking is critical:**

```
Customer places order → Status: PENDING
↓
Cashier A clicks "Pick Order #ORD-001"
↓
System atomically assigns order to Cashier A → Status: ASSIGNED
↓
Cashier B sees: "Order already assigned"
```



Critical Concurrency Requirement: Two cashiers clicking "Pick" at nearly the same time must not both acquire the order. The **backend/database must atomically assign the order** using database transactions and locking mechanisms. This cannot be solved with frontend JavaScript alone.

7.4 Order States

State	Description	Transition
PENDING	Order placed by customer, awaiting cashier pickup	Initial state after order
ASSIGNED	Cashier has picked the order	When cashier clicks "Pick"
READY	Order prepared, awaiting customer pickup/delivery	When cashier marks as ready
COMPLETED	Order fulfilled	When customer receives order
CANCELLED	Order cancelled	Admin or system action

7.5 Future Payment Integration

Phase 1 (Current): Customer places order without payment. Cashier processes and fulfills.

Phase 2 (Future): Payment API integration will add:

Order → Payment → Payment Confirmed → Branch/Cashier Processing → Ready → Completed

The database will be designed from the beginning so that adding payment later **does not require rebuilding** the ordering system.

Incomplete Orders

An important rule to define: **What happens when a cashier picks an order but never completes it?** The Admin should be able to reassign it after a certain period. This business rule will be formalized during detailed design.

8. Cashier Accountability

This is a **core requirement** — the system must provide cashier **accountability**, not just sales monitoring.

8.1 Accountability Report

For any selected date, the Branch Admin can view a comprehensive accountability table for each cashier:

Cashier	Sales	Expected Cash	Actual Cash	Difference	Status
Cashier A	TZS 450,000	450,000	450,000	0	Balanced
Cashier B	TZS 380,000	380,000	365,000	-15,000	 Loss
Cashier C	TZS 520,000	520,000	525,000	+5,000	 Surplus

What the Admin Can Determine

- Which cashier handled which transactions
- Total sales by each cashier
- Expected cash amount
- Actual cash amount received
- Cash shortage/loss amount
- Cash surplus amount
- Expenses entered by the cashier (if permitted)
- Refunds and voided transactions
- Damaged or missing stock attributable to a cashier
- The **exact transactions** contributing to the difference

8.2 Discrepancy Reasons

Critical Rule: The system must **not** simply label every difference as "cashier loss." The system must record the *reason* for every discrepancy.

Recognized discrepancy reasons include:

Reason	Description
Approved Expense	Cashier legitimately spent cash on an approved operational expense
Refund	Cash refund given to a customer for a returned item
Discount	An authorized discount was applied
Genuine Cash Shortage	Unexplained shortage — actual loss
Surplus	More cash received than expected
Damaged Stock	Product damage attributable to the cashier
Missing Stock	Stock unaccounted for and attributable to the cashier

Requirement: Branch Admin → Cashier Performance & Loss Accountability
→ Identify the cashier, date, transaction(s), amount, and reason for every recorded discrepancy/loss.

8.3 Cashier Expenses

Cashiers can create expenses that are tracked and attributed to them.

Required Expense Information

Field	Required	Example
Expense Category	Yes	Electricity
Amount	Yes	TZS 50,000
Description	Yes — Mandatory	Payment of electricity bill for Kariakoo branch for August 2026
Date	Auto-recorded	August 20, 2026
Entered By	Auto-recorded	Cashier A

Important: The cashier must provide a **detailed explanation** for every expense — not simply "Electricity: 50,000." This makes the expense useful for

auditing and affects profit calculations.

9. Financial Management

9.1 Profit Calculation

The system calculates profit correctly — not simply `Selling Price - Buying Price` . It must consider:

- Actual stock movements
- Discounts applied
- Expenses incurred
- Returns/refunds
- Damaged/missing stock

Profit Formula (Conceptual)

```
Revenue = Sum of all completed sales
COGS = Sum of (buying cost × quantity sold)
Gross Profit = Revenue - COGS
Expenses = Sum of all recorded expenses (by category)
Net Profit = Gross Profit - Expenses
```

9.2 Expense Management

Expenses are categorized and tracked:

Category	Examples
Electricity	Monthly utility bills
Water	Water utility bills
Transport	Delivery, travel expenses
Cleaning	Branch cleaning services
Packaging	Bags, boxes, wrapping
Other	Any other operational expense

Every expense records: category, amount, detailed description, date, and the person who entered it (cashier or admin).

9.3 Financial History

The Branch Admin can investigate the financial history of their branch by selecting a date range:

Daily View Example — Kariakoo Branch, August 20, 2026

Metric	Value
Total Sales	TZS 1,200,000
Cost of Goods Sold	TZS 700,000
Gross Profit	TZS 500,000
Expenses	TZS 80,000
Net Profit	TZS 420,000
Number of Transactions	15
Number of Products Sold	22
Cashier Performance	See accountability report
Stock Received	30 units
Stock Remaining	145 units

This can be viewed across: **Daily** → **Weekly** → **Monthly** → **Yearly**

10. Reporting

10.1 Report Types

Report	Frequency	Description
Sales Report	Daily / Weekly / Monthly / Yearly	Transactions, revenue, items sold
Profit Report	Daily / Weekly / Monthly / Yearly	Gross profit, net profit, margins
Expense Report	Daily / Weekly / Monthly / Yearly	All expenses by category
Stock Report	On-demand	Stock levels, movements, adjustments
Cashier Performance	Daily / Weekly / Monthly	Sales, accountability, discrepancies
Product Performance	Daily / Weekly / Monthly / Yearly	Top sellers, slow movers, margins

10.2 Reporting Scope

Rule:

- **Branch Admin reports** → restricted to own branch only
- **Super Admin reports** → company-wide, all branches, with drill-down into individual branches

11. User Profile Management

11.1 Super Admin Profile

Action	Allowed
View profile	
Edit name	
Edit phone	
Edit email	
Change password	
Upload profile picture	 (not required)
Register GPS location	 (not applicable)

11.2 Branch Admin Profile

Action	Allowed
View profile	
Edit name	
Edit phone	
Edit email	
Change password	
Change profile picture	
Register/update branch GPS	
Edit business profile info	
View cashier passwords	

11.3 Cashier Profile

Action	Allowed
View profile	
Edit name	
Edit phone	
Edit email	
Change profile picture	
Change password	
Change branch assignment	
Change approval status	
Change role	

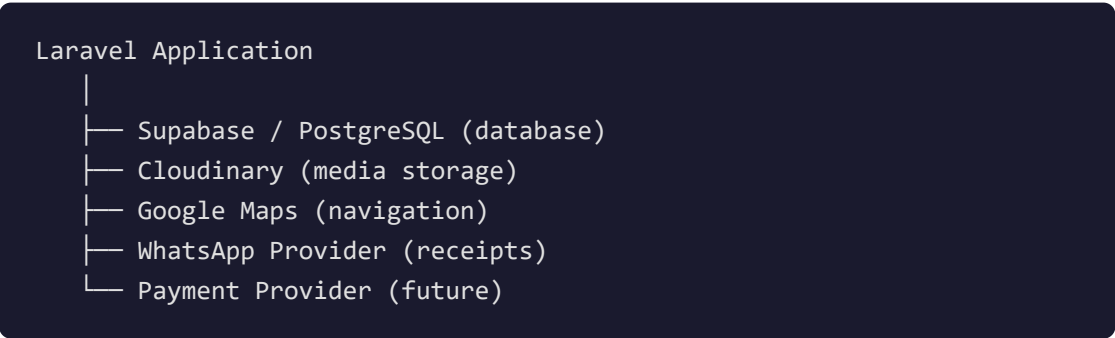
12. Technology Stack

12.1 Core Technologies

Component	Technology	Purpose
Backend / Application	Laravel (PHP)	Application framework, routing, controllers, business logic
Database	Supabase (PostgreSQL)	Relational database, authentication, real-time subscriptions
File / Image Storage	Cloudinary	Product images, profile pictures, media files
Authentication	Laravel Auth + OTP	Login, registration, email verification
Maps / Directions	Google Maps API	Branch navigation for customers
WhatsApp Receipts	WhatsApp Business API	Automated PDF receipt delivery (provider TBD)
Payments	<i>Not yet integrated</i>	To be added in Phase 2

12.2 External Services

All external services are designed as **loosely coupled** modules:



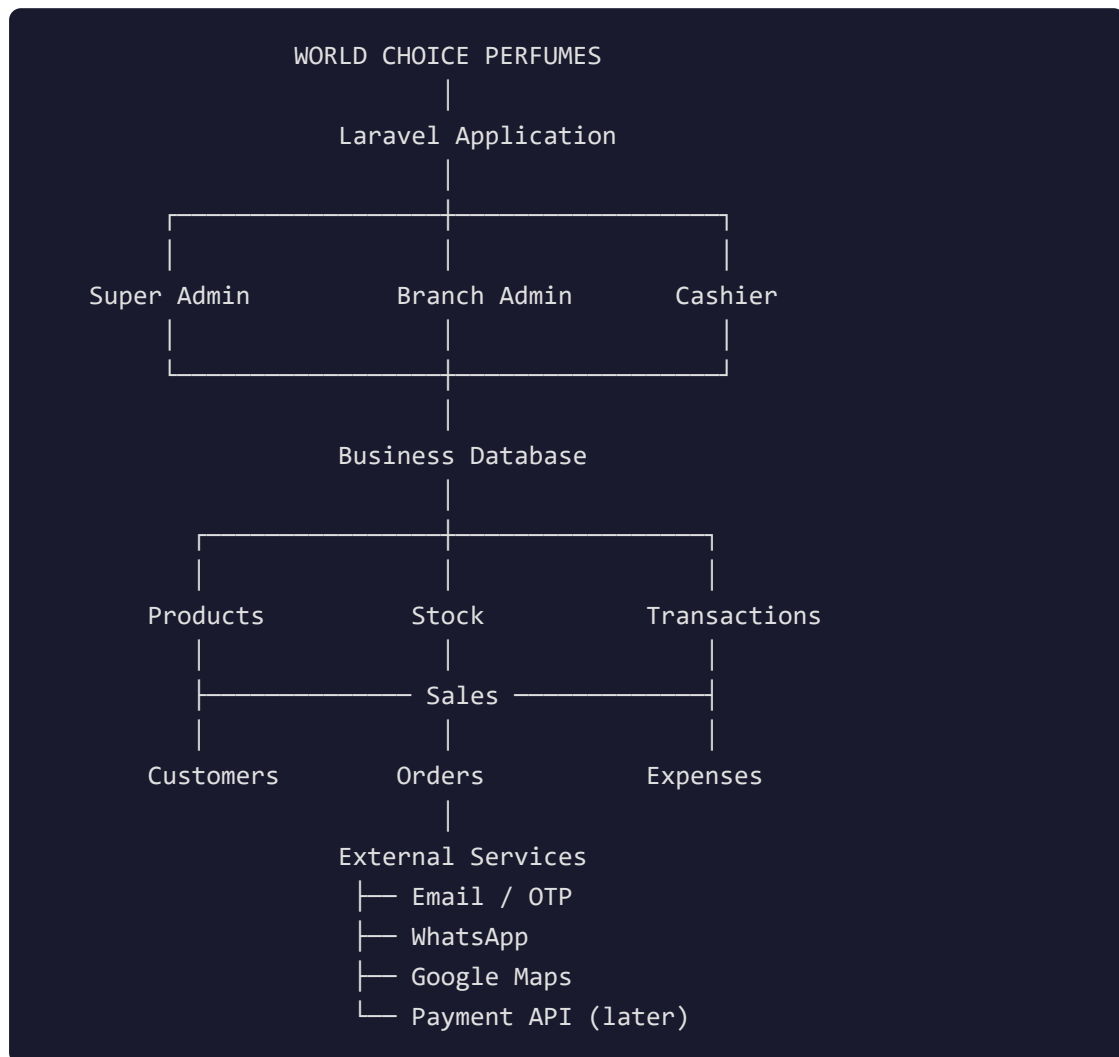
Architectural Principle: If you later change the WhatsApp provider or payment provider, it should not require tearing apart the entire Laravel application. Each integration is a swappable module.

12.3 Architectural Principles for Implementation

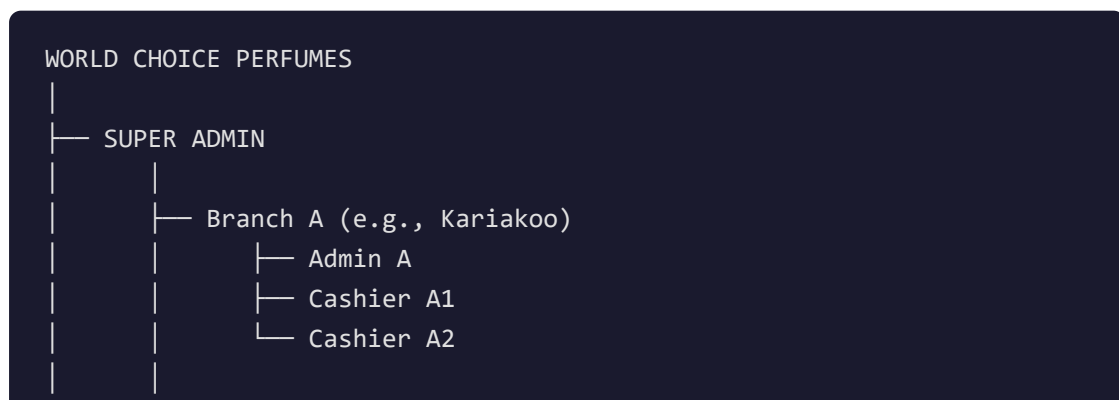
- Proper Laravel architecture and conventions
- Strong authentication and authorization
- Role-based and branch-based access control
- Database transactions for all critical operations
- Concurrency protection for orders and stock
- Stock integrity through atomic operations
- Financial/accounting accuracy
- Audit logs for all significant actions
- Secure API design
- Validation and sanitization
- Proper exception handling
- Queues and background jobs where appropriate
- Notifications (email, in-app)
- PDF receipt generation
- Database indexing for performance
- Secure file uploads
- Rate limiting
- Logging and monitoring
- Backup and recovery procedures
- Production deployment and VPS hardening
- Testing (unit, feature, integration)
- Maintainable code structure
- Proper database normalization

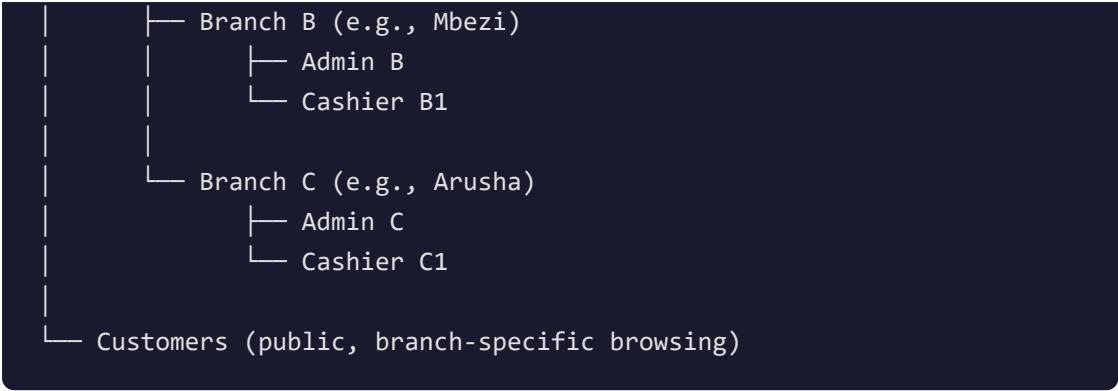
13. System Architecture

13.1 High-Level Architecture



13.2 Organizational Hierarchy





13.3 Data Isolation Model

Role	Data Access	Enforcement Level
Super Admin	All branches, all data	System-level
Branch Admin	Only assigned branch data	Backend + Database
Cashier	Own transactions within assigned branch	Backend + Database
Customer	Public product info for selected branch only	API-level

Security Requirement: Data isolation must be enforced at the **backend/database level**, not merely by hiding menu items in the frontend. A user bypassing the UI (e.g., through API calls) must still be restricted to their authorized data.

14. Database Design Requirements

14.1 Core Entities

The database will include (at minimum) the following tables:

Entity	Description
users	All system users (Super Admin, Branch Admin, Cashier) with roles
roles	User roles (super_admin, branch_admin, cashier)
branches	Branch details, location, assigned admin
products	Product catalog (name, description, brand, category)
product_images	Cloudinary references for product images
branch_stock	Stock levels per product per branch with buying cost and selling price
stock_movements	Stock entries, sales, returns, adjustments, damage records
customers	Customer contact information captured during sales
sales	Sale transactions with cashier, branch, total, timestamp
sale_items	Individual products within each sale
orders	Customer orders (branch-specific) with status tracking
order_items	Products within each order
expenses	Expenses with category, amount, description, attributed user
cashier_accounts	Cashier accountability records (daily expected vs actual)
discrepancies	Discrepancy records with reason codes and linked transactions
otp_records	OTP tokens for email verification
audit_logs	System-wide audit trail for all significant actions
notifications	User notifications

14.2 Key Relationships

```
Branch — has many —> Cashiers
Branch — has many —> Stock entries
Branch — has many —> Sales
Branch — has many —> Orders
Branch — has one —> Admin

Cashier — has many —> Sales
Cashier — has many —> Expenses
Cashier — has many —> Stock entries (entered by)
Cashier — has many —> Accountability records

Sale — has many —> Sale Items
Sale — belongs to —> Cashier
Sale — belongs to —> Branch
Sale — linked to —> Customer

Order — has many —> Order Items
Order — belongs to —> Branch
Order — assigned to —> Cashier (optional)

Product — stocked in —> Branch Stock (with branch-specific price & quantity)
Product — has many —> Product Images (Cloudinary)
```

15. Security Requirements

#	Requirement	Priority
1	Data isolation enforced at backend/database level	Critical
2	Role-based access control (RBAC) for all endpoints	Critical
3	Branch-based data filtering in all queries	Critical
4	Database transactions for stock operations (atomic)	Critical
5	Database-level locking for concurrent order assignment	Critical
6	OTP verification for all account registrations	High
7	Secure password hashing (bcrypt/argon2)	Critical
8	CSRF protection on all forms	Critical
9	Input validation and sanitization	High
10	SQL injection prevention (parameterized queries)	Critical
11	XSS prevention	High
12	Rate limiting on authentication endpoints	High
13	Secure file upload validation (Cloudinary)	Medium
14	Audit logging for all significant actions	High
15	HTTPS enforcement in production	Critical
16	Environment variable security (no secrets in code)	Critical
17	API token/session management	High
18	Exception handling (no sensitive data in error messages)	High

16. Business Rules Summary

#	Rule
BR-01	Super Admin has access to the entire company; Branch Admin only to their branch
BR-02	Cashier accounts require Admin approval before login is permitted
BR-03	Cashiers cannot change selling prices — prices are set by the Branch Admin
BR-04	Every sale reduces stock atomically — no negative stock allowed
BR-05	Order assignment must be atomic — no double-pick by two cashiers
BR-06	Every expense must include a detailed description for auditing
BR-07	Every discrepancy must record a reason (not just "loss")
BR-08	Automatic WhatsApp receipt sent after every completed sale
BR-09	Branch GPS coordinates are set by Admin and reusable by customers
BR-10	Customer locations are NOT stored in the database
BR-11	Product prices and availability are branch-specific
BR-12	Reports are scoped to the user's authorized branches only
BR-13	External services must be loosely coupled (swappable providers)
BR-14	Payment integration will be added later — core system must not depend on it

BR-15	Admin can edit branch GPS coordinates at any time (not locked to registration)
-------	--

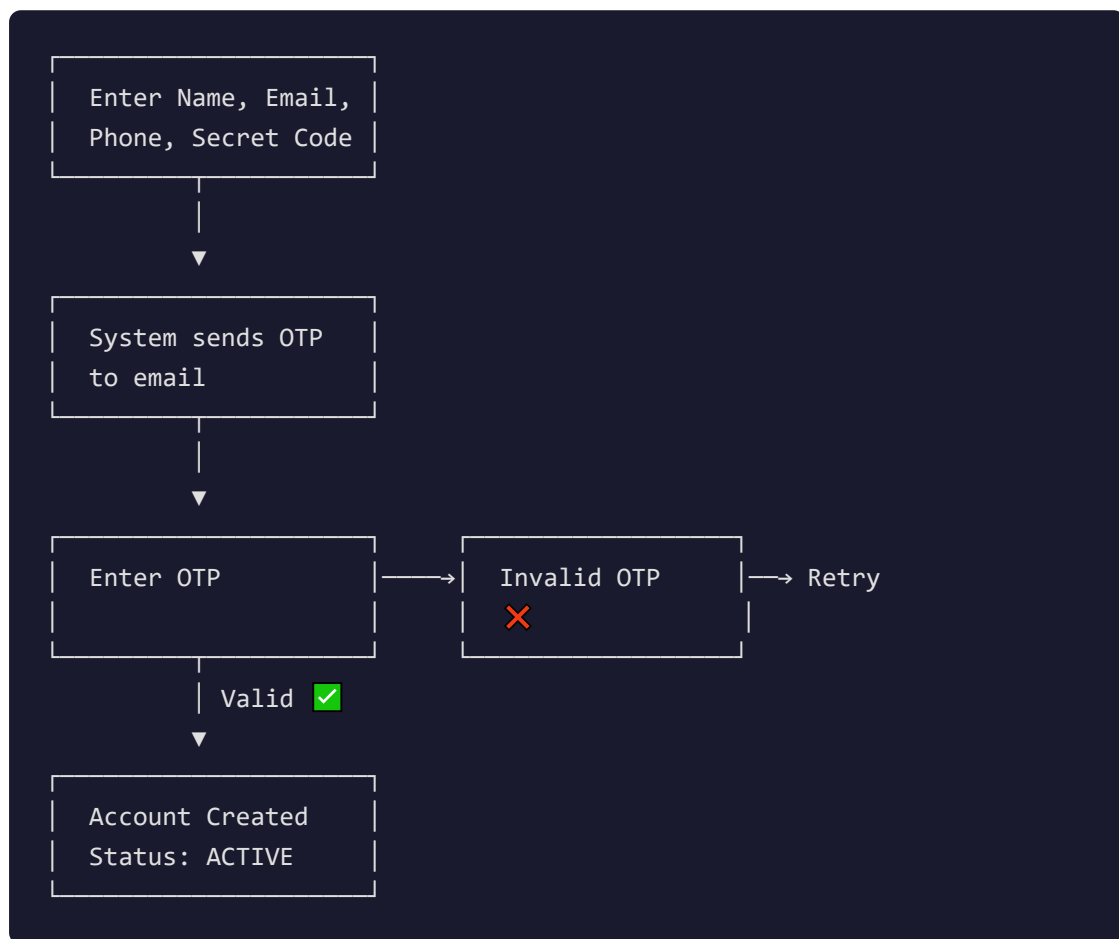
17. Future Enhancements

Feature	Phase	Description
Payment API Integration	Phase 2	Mobile money, card payments, or other payment methods
Discount Authorization	Phase 2	Authorized discounts with Admin approval workflow
Advanced Customer CRM	Phase 2	Customer profiles, purchase history analytics, marketing
Multi-Language Support	Phase 3	Swahili and English interfaces
Mobile Application	Phase 3	Native or cross-platform mobile app
Delivery Management	Phase 2	Order delivery tracking, delivery personnel assignment
Inventory Alerts	Phase 2	Low-stock notifications, automatic reorder suggestions
Advanced Analytics	Phase 3	Business intelligence dashboards, trend analysis

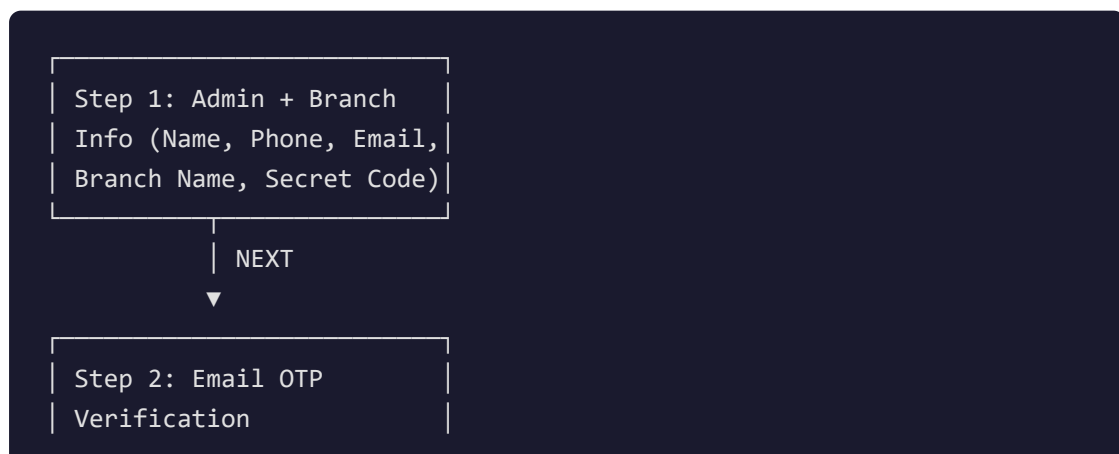
18. Appendices

Appendix A: Registration Flow Diagrams

A.1 Super Admin Registration

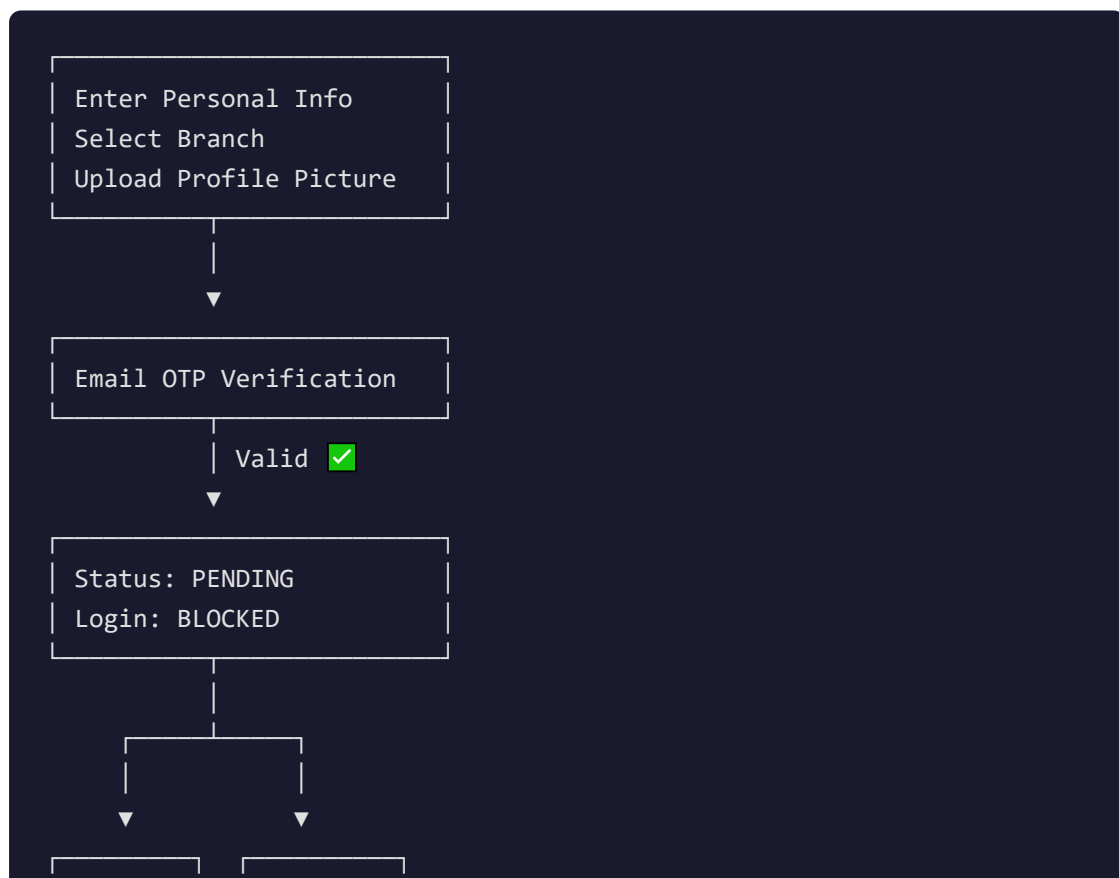


A.2 Branch Admin Registration



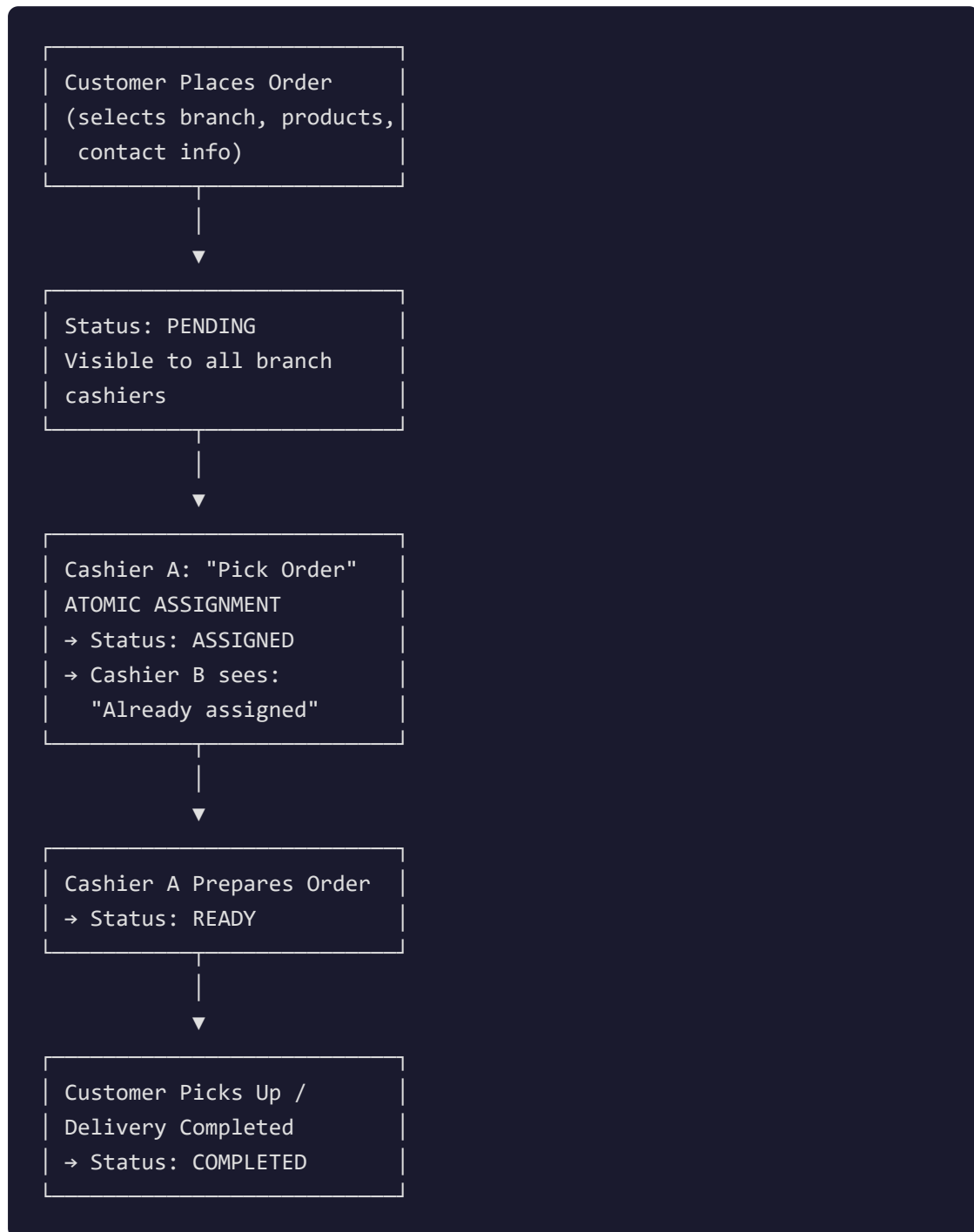


A.3 Cashier Registration



Admin	Admin
Approves	Rejects
→ 	→ 
APPROVED	REMOVED
Can Login	

Appendix B: Order Workflow Diagram



Appendix C: Permissions Matrix

Feature	Super Admin	Branch Admin	Cashier	Customer
Create/manage branches	✓	✗	✗	✗
Assign Admin to branch	✓	✗	✗	✗
View all branches	✓	✗	✗	✗
View own branch data	✓	✓	✓*	✗
Approve/reject cashiers	✓	✓	✗	✗
Enter stock	✓	✓	✗	✗
Set selling prices	✓	✓	✗	✗
Make sales	✗	✗	✓	✗
Create expenses	✗	✓	✓	✗
View expense details	✓	✓	Own only	✗
View financial reports	All branches	Own branch	✗	✗
Cashier accountability	✓	✓	✗	✗
Browse products	✗	✗	✗	✓
Place orders	✗	✗	✓	✓
Pick/fulfill orders	✗	✗	✓	✗
View audit logs	✓	✗	✗	✗
System settings	✓	✗	✗	✗

* Cashier sees only own operational records within assigned branch.

World Choice Perfumes — System Specification v1.0

Document generated: August 21, 2026

Instagram: @world_choice_parfum | Phone: +255 710 603 637 | TikTok: Kessy Mohammed02